Cine - **Manual técnico**

13/04/2026

Clasificación: publico

Elaborado por: Angel Gael Gaytan Bustamante

Fernanda Bracho Guitron

Norma de referencia: IEEE 1016-2009

1.Control de cambios

Versión	Fecha	Descripción del cambio	Autor(es)
1.0	[04/04/2026]	Creación inicial del documento	Fernanda Bracho Ángel Gaytan
2.0	[13/04/2026]	Redacción final del documento	Ángel Gaytan Fernanda Bracho

2.Firmas de aprobación

Elaboraron	Reviso	Autorizo
Angel Gaytan Fernanda Bracho	Irving Mendoza	Irving Mendoza
Desarrolladores	Cliente	Cliente
[04/04/2026]	[15/04/2026]	[15/04/2026]

3.Introduccion

3.1 Propósito del documento

Este manual tiene como propósito ser la referencia absoluta para arquitectos, desarrolladores, administradores de sistemas y todo aquel que quiera entender cómo funciona técnicamente el sistema desarrollado en el dicho documento.

3.2 Alcance del diseño técnico

El presente diseño técnico especifica la arquitectura, las estructuras de datos y las interfaces de usuario del sistema de gestión cinematográfica. El alcance abarca la definición de las entidades de software, su organización en paquetes y los mecanismos de interacción entre el frontend (interfaz gráfica) y el backend (lógica de negocio).

3.2.1 Organización del Diseño (Package Structure)

El sistema se desglosa en dos capas principales siguiendo una arquitectura desacoplada:

- **Capa de Lógica de Negocio (Backend) (com.mycompany.proyecto):** Define las estructuras de datos dinámicas y los modelos de información. Incluye entidades como Pelicula, CarteleraGeneral, GestionCines, Cine y Horario, así como la gestión de estructuras de datos especializadas: ListaPeliculas, ColaDulceria, PilaHistorial,NodoCola, NodoPelicula.
- **Capa de Presentación(Frontend) (ui):** Implementa la experiencia del usuario mediante componentes Swing los cuales se conectan con las entidades en el package **com.mycompany.proyecto**. Tiene una pantalla de selección de cine y login que funciona mediante PantallaSeleccioncine y PantallaLogin. Incluye módulos especializados para diferentes perfiles de acceso: PantallaAdmin para gestión operativa y PantallaCliente para consumo de servicios.

3.2.2 Elementos de Diseño Críticos

1. **Manejo de Estructuras Dinámicas:** El diseño especifica la implementación manual de nodos (NodoPelicula, NodoCola) para la manipulación de información en memoria, optimizando el uso de recursos sin dependencias de frameworks externos de persistencia.
2. **Algoritmia de Procesamiento:** Se incluye el diseño del algoritmo Quicksort dentro de la clase CarteleraGeneral, permitiendo el ordenamiento concurrente de registros de cartelera bajo múltiples criterios (fecha y nombre).

3. Interfaces de Usuario (GUI): El diseño contempla la creación de interfaces visuales con layouts jerárquicos (BoxLayout, GridBagLayout) y el renderizado de elementos gráficos personalizados (degradados y fuentes Unicode).

3.2.3 Límites y Restricciones del Diseño

- **Consistencia de Datos:** La integridad de la información se gestiona a través de la clase controladora GestionCines, que actúa como el punto único de acceso para las sucursales de diferentes estados y municipios.
- **Restricciones de Implementación:** El diseño está orientado exclusivamente al lenguaje de programación Java, utilizando bibliotecas estándar de la JVM para garantizar la portabilidad del sistema en entornos de escritorio.

3.3 Referencias

- IEEE 1016-2009 - IEEE Standard for Information Technology - Systems Design - Software Design Descriptions
- Rubrica de evaluación de proyecto – Irving Mendoza

4.Descripción General del Sistema

4.1 Perspectiva del producto

El sistema de Cine se define como una aplicación de escritorio independiente y autocontenida. No es parte de una suite preexistente, sino que funciona como una entidad centralizada para la administración de sucursales de cine en un entorno local.

4.1.1 Contexto del Sistema

El sistema actúa como el puente operativo entre la base de datos lógica (estructuras de datos en memoria) y los diferentes perfiles de usuario. Se integra en el flujo de trabajo de una cadena de cines distribuida geográficamente en estados y municipios, permitiendo la interoperabilidad entre los módulos de administración y atención al cliente.

4.1.2 Interfaces de Usuario

La interacción se realiza a través de una interfaz gráfica de usuario (GUI) desarrollada en Java Swing. El diseño sigue un esquema de navegación basado en CardLayout y menús laterales dinámicos, permitiendo una transición fluida entre los módulos de búsqueda, cartelera y dulcería. El sistema utiliza fuentes Unicode y renderizado de emojis para mejorar la experiencia visual en sistemas operativos modernos.

4.1.3 Interfaces de Hardware y Entorno Operativo

- **Plataforma:** El sistema es multiplataforma gracias a la **Java Virtual Machine (JVM)**, operando principalmente en entornos Windows, macOS y Linux.
- **Recursos:** Requiere un entorno con soporte para gráficos Java2D para el renderizado de degradados y componentes personalizados.
- **Hardware:** No requiere periféricos especializados, operando con dispositivos de entrada estándar (teclado y mouse) y monitores con resolución mínima de **1024x768** píxeles.

4.1.4 Interfaces de Software (Sistemas Externos)

- **Persistencia:** En la fase actual, el sistema no depende de un Sistema de Gestión de Bases de Datos (RDBMS) externo, integrando su propia lógica de persistencia volátil mediante la clase GestionCines.

4.1.5 Jerarquía de Usuarios

El producto se integra en el contexto operativo mediante dos niveles de acceso:

1. **Perfil Administrador:** Usuarios con privilegios para modificar la lógica del negocio (Bajas de horarios, gestión de salas).
2. **Perfil Cliente:** Usuarios finales que interactúan con los servicios de consulta, búsqueda y pedidos.

4.2 Funciones principales

El diseño del software se divide en módulos funcionales que interactúan entre sí para separar la lógica de presentación de la lógica de negocio y el procesamiento de datos.

- **Ordenamiento Dinámico:** Implementación del algoritmo **Quicksort** (clase CarteleraGeneral) para organizar películas por fecha o nombre en orden ascendente/descendente.
- **Búsqueda Multicriterio:** Lógica de filtrado en tiempo real por clasificación, género y nombre utilizando recorridos en listas ligadas (ListaPeliculas).
- **Gestión de Horarios:** Control de sesiones, salas y fechas mediante la entidad Horario.
- **Gestión de Pedidos (Dulcería):** Control secuencial de pedidos mediante una estructura de **Cola** (ColaDulceria), garantizando el orden de llegada de los clientes.
- **Historial de Consultas:** Registro de la actividad del usuario utilizando una **Pila** (PilaHistorial), permitiendo recuperar las últimas películas consultadas (LIFO).
- **Localización Geográfica:** Algoritmo de selección de sucursal más cercana operado a través de la clase GestionCines.
- **Autenticación de Usuarios:** Gestión de acceso administrativo con validación de credenciales y control de flujo de seguridad en PantallaLogin.
- **Mantenimiento de Sucursales:** Capacidad de modificar estados operativos y parámetros de los cines distribuidos por municipio.
- **Navegación Dinámica:** Uso de un contenedor CardLayout para permitir el intercambio de paneles de contenido sin abrir múltiples ventanas.
- **Renderizado Personalizado:** Dibujo de fondos degradados y componentes estilizados con soporte para fuentes enriquecidas (Emoji Unicode).

4.3 Características de los usuarios

Rol	Descripción	Permisos / Acceso
Administrador	Usuarios con privilegios para modificar la lógica del negocio (Bajas de horarios, gestión de salas).	**Gestión de Inventario:** Modificación y baja de registros en la estructura ListaPeliculas. **Control de Funciones:** Administración de la clase Horario, permitiendo habilitar o deshabilitar sesiones según la disponibilidad de las salas. **Autenticación:** Es el único perfil con acceso al módulo PantallaLogin, el cual cuenta con mecanismos de seguridad por intentos fallidos. **Supervisión Operativa:** Capacidad de visualizar el estado interno de las sucursales gestionadas por la clase GestionCines.

4.4 Restricciones generales

Estas restricciones limitan las opciones de diseño y definen el marco operativo obligatorio para el desarrollo del sistema.

Arquitectura Monolítica: El sistema está diseñado para ejecutarse como una aplicación de escritorio única (Standalone), lo que limita su escalabilidad en entornos distribuidos o web sin una reingeniería previa.

Gestión de Memoria: Al utilizar estructuras de datos manuales (ListaPeliculas, PilaHistorial, ColaDulceria) en lugar de una base de datos persistente, la capacidad de almacenamiento está restringida por la memoria RAM disponible de la Java Virtual Machine (JVM).

Interfaz Gráfica: Se utiliza estrictamente la biblioteca **Java Swing**, lo que restringe el diseño a componentes pesados (Heavyweight) y ligeros de Java, limitando el uso de aceleración por hardware avanzada.

4.5 Suposiciones y dependencias

Este apartado describe los factores que se han considerado como constantes y los elementos externos de los cuales depende el sistema para su éxito.

- **Suposiciones de Diseño:**
  - **Integridad de los Datos de Entrada:** Se asume que los datos proporcionados por el administrador (precios, horarios, nombres) son válidos y siguen el formato esperado, ya que el sistema cuenta con validaciones de tipo, pero no de veracidad lógica extrema.
  - **Persistencia Volátil:** Se asume que el ciclo de vida de los datos es igual al ciclo de vida de la ejecución del programa. No se asume la recuperación de datos tras un cierre inesperado del sistema (Crash) en esta versión.
  - **Compatibilidad de Fuentes:** Se asume que el sistema operativo anfitrión posee instaladas fuentes compatibles con símbolos Unicode para la correcta visualización del menú lateral y los iconos.
- **Dependencias Externas:**
  - **Java Virtual Machine (JVM):** El sistema depende totalmente de la disponibilidad y correcta configuración de la JVM en el entorno operativo.
  - **Bibliotecas del Proyecto:** El correcto funcionamiento depende de la integridad del paquete com.mycompany.proyecto, específicamente de las clases controladoras como GestionCines, que orquestan la creación de los objetos de negocio.
  - **Lógica de Negocio:** El módulo de interfaz (ui) depende estrictamente de los métodos públicos expuestos por la clase Cine para realizar búsquedas, filtrados y gestión de historial.

5.Arquitectura del sistema

5. Arquitectura del Sistema

El sistema está diseñado bajo un modelo de Arquitectura Multicapa, lo que permite una separación clara entre la interfaz de usuario, la lógica de procesamiento y el almacenamiento de datos. Este enfoque facilita el mantenimiento y asegura que los algoritmos de estructuras de datos operen de forma independiente a la visualización.

5.1 Descripción de las Capas

1. Capa de Presentación (Interfaz de Usuario): Construida con la biblioteca Java Swing, se encarga de capturar las interacciones del usuario (clics, ingresos de texto) y desplegar la información procesada. No contiene lógica de negocio; su única función es delegar las peticiones al controlador.
2. Capa de Negocio (Controlador y Lógica): Representada principalmente por la clase Cine. Es el "cerebro" del sistema que coordina el flujo de información. Aquí se ejecutan los algoritmos críticos como la detección de solapamiento de horarios, el cálculo de rutas y la gestión de las estructuras dinámicas (ListaPeliculas, PilaHistorial, ColaDulceria).
3. Capa de Persistencia (Acceso a Datos): A diferencia de los sistemas tradicionales que usan SQL, este sistema implementa una persistencia basada en Archivos Planos (.txt). El acceso a datos es gestionado mediante flujos de entrada/salida (BufferedReader/PrintWriter), permitiendo que la información se mantenga de forma permanente en el disco duro de la sucursal.

5.1.1 Principios de Diseño Aplicados

- Encapsulamiento: Cada clase (como CarteleraGeneral o Pelicula) protege sus datos internos y solo expone los métodos necesarios para su manipulación.
- Desacoplamiento: La lógica de las estructuras de datos (como el Quicksort en CarteleraGeneral) es independiente de la interfaz gráfica, lo que permitiría cambiar la ventana de Swing por una interfaz de consola o web sin modificar el algoritmo de ordenamiento.
- Persistencia Local: El diseño está optimizado para funcionamiento *offline*, garantizando que cada cine sea autónomo en la gestión de sus propios recursos de cartelera.

5.2 Descripción de componentes

5.2.1 Componente - CarteleraGeneral

Atributo	Descripción
Nombre	com.mycompany.proyecto.CarteleraGeneral
Propósito	Consolidar la información de películas y horarios en una estructura tabular optimizada para el procesamiento de datos y visualización.
Responsabilidades	1. **Modelado de Datos**: Almacenar atributos de nombrePelicula, sala, hora y fecha. 2. **Algoritmia**: Implementar el ordenamiento **Quicksort** mediante el esquema de partición de Lomuto. 3. **Normalización**: Generar claves de comparación cronológicas a través del método clave.
Interfaces	1. **Entrada**: Recibe arreglos de objetos CarteleraGeneral desde la lógica del Cine. 2. **Salida**: Provee datos ordenados al DefaultTableModel de la interfaz de usuario.
Dependencias	1. **Lógica**: Utiliza los datos de las clases Pelicula y Horario. 2. **UI**: Es invocado directamente por PantallaCliente para actualizar la vista de la cartelera.
Tecnología	**Java SE**: Implementación de lógica algorítmica y manipulación de arreglos de objetos

5.2.2 Componente -Cine

Atributo	Descripción
Nombre	com.mycompany.proyecto.Cine
Propósito	Funcionar como la entidad gestora de una sucursal específica, coordinando la persistencia de datos, la lógica de horarios y los servicios al cliente.
Responsabilidades	1. **Persistencia**: Cargar y guardar la información de la sucursal en archivos de texto plano (.txt). 2. **Gestión de Funciones**: Validar el solapamiento de horarios en salas y dar de alta/baja películas. 3. **Control de Servicios**: Administrar la cola de la dulcería y el historial de consultas mediante estructuras dinámicas.
Interfaces	1. **Entrada de Usuario**: Recibe peticiones desde PantallaCliente y PantallaAdmin para altas, bajas y consultas. 2. **Acceso a Datos**: Interactúa con la clase ListaPeliculas para operaciones de búsqueda binaria.
Dependencias	1. **Modelos**: Pelicula, Horario, NodoPelicula. 2. **Estructuras**: ListaPeliculas, ColaDulceria, PilaHistorial. 3. **I/O**: Depende de las clases File, BufferedReader y PrintWriter para el manejo de archivos.
Tecnología	**Java SE**: Uso de manejo de excepciones, flujos de archivos (I/O) y lógica de comparación aritmética para tiempos.

5.2.3 Componente – ColaDulceria

Atributo	Descripción
Nombre	com.mycompany.proyecto.ColaDulceria
Propósito	Gestionar el flujo de pedidos de productos de consumo mediante una estructura de datos lineal tipo FIFO (First-In, First-Out).
Responsabilidades	1. **Inserción (Push)**: Agregar nuevos pedidos al final de la cola manteniendo la referencia del nodo fin. 2. **Extracción (Pop)**: Despachar el pedido más antiguo (cabeza) y actualizar las referencias de la estructura. 3. **Formateo de Datos**: Transformar los códigos numéricos de combos en descripciones textuales para las interfaces de usuario.
Interfaces	1. **Módulo Cliente**: Recibe solicitudes de inserción desde el panel de Dulcería. 2. **Módulo Administrativo**: Provee arreglos de Strings (getListaParaAdmin) para la visualización del estado de pedidos
Dependencias	1. **Estructurales**: Clase NodoCola (unidad básica de almacenamiento). 2. **Contextuales**: Es una propiedad integrante de la clase Cine.
Tecnología	**Java SE**: Uso de gestión de punteros (referencias) para la creación de una lista ligada simple con comportamiento de cola.

5.2.4 Componente – GestionCines

Atributo	Descripción
Nombre	com.mycompany.proyecto.GestionCines
Propósito	Orquestar la red global de sucursales cinematográficas y resolver problemas de optimización de rutas y distancias entre sedes.
Responsabilidades	1. **Inventario de Sedes**: Inicializar y mantener el arreglo global de objetos Cine para múltiples estados (Jalisco, Nuevo León, etc.). 2. **Cálculo de Rutas**: Implementar el algoritmo de **Floyd-Warshall** para encontrar los caminos mínimos en grafos de distancias. 3. **Localización**: Identificar la sucursal más cercana a la posición actual del usuario mediante el análisis de matrices de adyacencia.
Interfaces	1. **Módulo de Selección**: Provee el método seleccionar para cargar los datos específicos de una sucursal al iniciar la aplicación. 2. **Módulo de Consulta**: Expone la función cineMasCercano consumida por la interfaz del cliente.
Dependencias	1. **Composición**: Clase Cine (el componente gestiona una colección de estas instancias). 2. **Librerías Estándar**: java.util.Arrays para la manipulación y copia de matrices de distancias.
Tecnología	**Java SE**: Uso de estructuras multidimensionales, implementación de algoritmo Floyd-Warshall estructuras de control switch y lógica algorítmica de grafos.

5.2.5 Componente – Horario

Atributo	Descripción
Nombre	com.mycompany.proyecto.Horario
Propósito	Representar una instancia específica de proyección cinematográfica definiendo su ubicación física y temporal.
Responsabilidades	1. **Encapsulamiento**: Almacenar los datos de la sala (entero), la hora y la fecha (cadenas de texto). 2. **Integridad**: Servir como objeto de valor para la validación de solapamientos dentro de la lógica del Cine.
Interfaces	1. **Consumo Externo**: Es instanciado por la clase Cine durante la carga de archivos. 2. **Visualización**: Sus atributos son leídos por CarteleraGeneral para los algoritmos de ordenamiento.
Dependencias	**Relacional**: Es un componente atómico contenido dentro del arreglo de horarios de la clase Pelicula.
Tecnología	**Java SE**

5.2.6 Componente – ListaPeliculas

Atributo	Descripción
Nombre	com.mycompany.proyecto.ListaPeliculas
Propósito	Gestionar una colección dinámica de películas mediante una lista ligada simple, proporcionando capacidades de búsqueda y ordenamiento.
Responsabilidades	1. **Gestión Dinámica**: Implementar operaciones de inserción (agregar) y eliminación (eliminar) de nodos en memoria. 2. **Conversión de Estructura**: Transformar la lista ligada en arreglos estáticos (toArray) para compatibilidad con otros módulos. 3. **Búsqueda Optimizada**: Ejecutar **Búsqueda Binaria** sobre los datos, previo ordenamiento interno mediante Quicksort por nombre.
Interfaces	1. **Módulo de Datos**: Expone el método busquedaBinaria para la localización rápida de títulos por parte de la clase Cine. 2. **Administración**: Permite la actualización de la cartelera física a través de la eliminación de nodos por nombre.
Dependencias	1. **Estructurales**: Clase NodoPelicula y Clase Pelicula. 2. **Algorítmicas**: Implementación interna de Quicksort (quickSortNombre) para garantizar la pre-condición de la búsqueda binaria.
Tecnología	**Java SE:** Uso de referencias de memoria (punteros) para el encadenamiento de objetos y recursividad para algoritmos de ordenamiento.

5.2.7 Componente – NodoCola

Atributo	Descripción
Nombre	com.mycompany.proyecto.NodoCola
Propósito	Definir la estructura básica de almacenamiento para la implementación de la lista ligada que conforma la cola de la dulcería.
Responsabilidades	1. **Almacenamiento de Datos**: Resguardar el identificador numérico (int) correspondiente al tipo de combo solicitado. 2. **Enlace Estructural**: Mantener la referencia (*puntero*) al siguiente elemento en la secuencia de la cola.
Interfaces	1. **Consumo Interno**: Es utilizado exclusivamente por la clase ColaDulceria para las operaciones de encolar y desencolar.
Dependencias	**Relacional**: Es un componente de infraestructura para ColaDulceria.
Tecnología	**Java SE**: Clase de soporte para estructuras de datos dinámicas con gestión de referencias de memoria.

5.2.8 Componente – NodoPelicula

Atributo	Descripción
Nombre	com.mycompany.proyecto.NodoPelicula
Propósito	Actuar como el contenedor dinámico que permite el encadenamiento de objetos de tipo Pelicula en una lista ligada.
Responsabilidades	1. **Referenciación**: Mantener el enlace hacia el siguiente nodo de la estructura (siguiente). 2. **Encapsulamiento de Objeto**: Resguardar la instancia completa de la clase Pelicula dentro de la estructura lineal.
Interfaces	1. **Consumo Estructural**: Es instanciado y gestionado por la clase ListaPeliculas. 2. **Acceso a Datos**: Permite a los algoritmos de búsqueda y eliminación acceder al contenido de cada película mediante recorridos de nodos.
Dependencias	1. **Contenido**: Clase Pelicula. 2. **Contenedor**: Clase ListaPeliculas.
Tecnología	**Java SE**: Implementación de nodos mediante referencias de memoria para la creación de estructuras de datos no contiguas.

5.2.9 Componente – Pelicula

Atributo	Descripción
Nombre	com.mycompany.proyecto.Pelicula
Propósito	Representar la entidad principal del cine, centralizando los datos y su oferta de horarios disponibles.
Responsabilidades	1. **Gestión de Atributos**: Almacenar información técnica (director, productor, clasificación, duración y género). 2. **Control de Horarios**: Administrar un arreglo estático de objetos Horario, permitiendo la inserción (agregarHorario) y eliminación (eliminarHorario) de funciones. 3. **Formateo de Salida**: Proveer representaciones textuales y resúmenes de horarios mediante StringBuilder para su despliegue en la interfaz de usuario.
Interfaces	1. **Módulo de Lógica**: Es contenido y gestionado por ListaPeliculas. 2. **Módulo de Vista**: Provee datos formateados a través del método obtenerHorariosString para los paneles de consulta del cliente.
Dependencias	1. **Composición**: Clase Horario (mantiene una colección interna de hasta 10 instancias). 2. **Infraestructura**: Utilizada por Cine para las operaciones de persistencia en archivos.
Tecnología	**Java SE**

5.2.10 Componente – PilaHistorial

Atributo	Descripción
Nombre	com.mycompany.proyecto.PilaHistorial
Propósito	Almacenar el registro de navegación y consultas de películas del usuario mediante una estructura de datos lineal pila tipo LIFO (Last-In, First-Out).
Responsabilidades	1. **Apilamiento (Push)**: Registrar el nombre de la película consultada en el tope de la estructura. 2. **Desapilamiento (Pop)**: Recuperar y eliminar el último elemento registrado. 3. **Visualización Inversa**: Desplegar los datos desde el elemento más reciente
Interfaces	1. **Módulo Cliente**: Invocado por Cine.agregarHistorial cada vez que se selecciona una película para ver detalles. 2. **Capa de Presentación**: Provee la información para el panel de "Historial" en la interfaz de usuario.
Dependencias	**Contextuales**: Funciona como un atributo de estado persistente dentro de la instancia de la clase Cine.
Tecnología	**Java SE**: Implementación de una pila (Stack) basada en un arreglo estático de Strings con puntero de gestión (tope).

5.2.11 Componente – Proyecto

Atributo	Descripción
Nombre	com.mycompany.proyecto. Proyecto
Propósito	Actuar como el punto de entrada principal del software, encargándose del arranque del sistema y la configuración del entorno visual.
Responsabilidades	1. **Inicialización de Estilo**: Configurar el *Look and Feel* (Nimbus) para garantizar una interfaz moderna y uniforme. 2. **Instanciación Maestra**: Crear la instancia única de GestionCines que persistirá durante toda la ejecución. 3. **Lanzamiento de UI**: Ejecutar el hilo de despacho de eventos de AWT (invokeLater) para abrir la ventana de inicio.
Interfaces	1. **Sistema Operativo**: Interfaz de línea de comandos que invoca el método main. 2. **Capa de Presentación**: Inyecta la lógica de gestión en PantallaSeleccioncine.
Dependencias	1. **Controlador**: GestionCines. 2. **Interfaz**: ui.PantallaSeleccioncine. 3. **Framework**: javax.swing.UIManager.
Tecnología	**Java SE**: Uso de métodos estáticos, manejo de excepciones para la configuración del entorno y programación concurrente básica para la GUI.

6.Stack Tecnológico

6.1 Resumen de tecnologías

Capa	Tecnología	Versión	Propósito
Frontend	Java Swing / AWT	8.0 o superior	Desarrollo de la interfaz gráfica de usuario (GUI) y manejo de eventos.
Backend	**Java SE (Standard Edition)**	JDK 17 / 21	Implementación de la lógica de negocio y algoritmos de estructuras de datos.
Base de datos	Flat Files (Archivos .txt)	NA	Almacenamiento persistente local mediante archivos de texto plano serializados manualmente.
Servidor / Nube	NA	NA	NA
Otros	Librería Standard I/O	1.0	Gestión de flujos de lectura y escritura (java.io) para el control de archivos.

6.2 Justificación de decisiones tecnológicas

El stack tecnológico seleccionado responde a la necesidad de crear un sistema ligero, autónomo y con alta eficiencia en el manejo de memoria. A continuación, se justifican las elecciones principales:

6.2.1 Java SE (Standard Edition)

- **Justificación:** Se eligió Java por su gestión automática de memoria (Garbage Collector) y su robustez en el manejo de Programación Orientada a Objetos (POO). Java permite un manejo de referencias más seguro para las listas ligadas y colas dinámicas del proyecto, reduciendo el riesgo de fugas de memoria (*memory leaks*). Además, que es el lenguaje impartido en la materia.

6.2.2 Java Swing / AWT

- **Justificación:** Swing se seleccionó debido a su familiaridad y estabilidad dentro del ecosistema Java. Al ser una librería nativa, no requiere dependencias externas pesadas ni configuraciones externas complejas, lo que garantiza que la interfaz gráfica sea uniforme y funcional en cualquier sistema operativo que cuente con un JRE (Java Runtime Environment).

6.2.3 Persistencia en Archivos Planos (.txt)

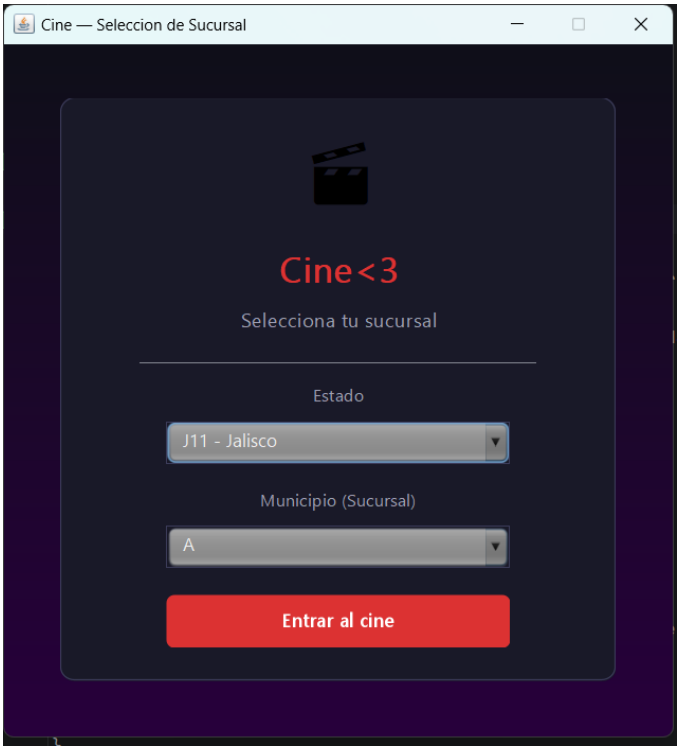
- **Justificación:** El uso de archivos .txt elimina la necesidad de instalar y configurar motores de bases de datos externos, facilitando la portabilidad del sistema (aplicación *Portable*). Además, permite validar el conocimiento en algoritmos de *parsing* y manejo de flujos de datos (File I/O), procesando la información directamente en estructuras de datos cargadas en RAM para una respuesta inmediata.

6.2.4 Algoritmos Manuales (Floyd-Warshall y Quicksort)

- **Justificación:** La implementación manual de estos algoritmos permite un control granular sobre la complejidad temporal y espacial del sistema. Floyd-Warshall fue implementado por obligación, aun así, fue de ayuda debido a que el mapa de cines es denso y pequeño (10x10), permitiendo pre-calcular todas las rutas posibles de forma eficiente. Quicksort se eligió por su rendimiento promedio de O(n \log n) (y por la necesidad de usar recursividad), ideal para el ordenamiento rápido de la cartelera antes de realizar búsquedas binarias.

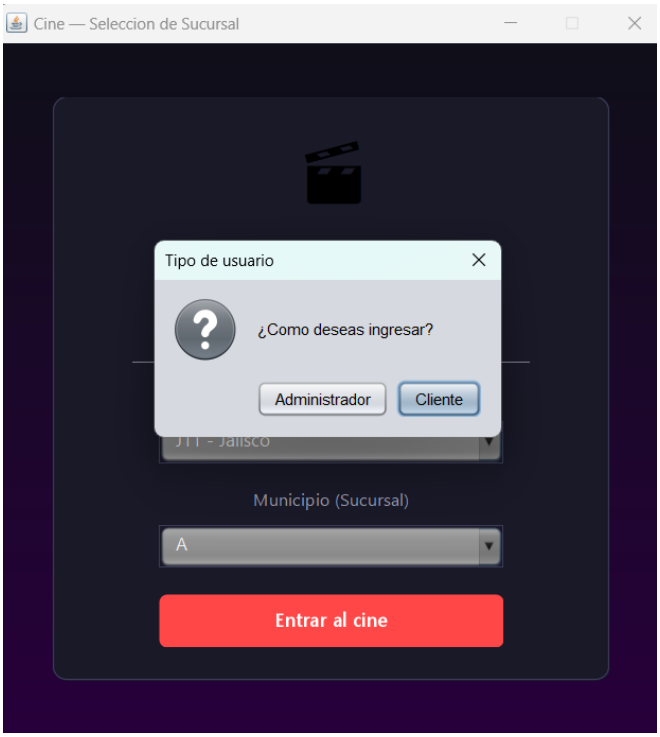
7.Diseño de interfaces

7.1 Interfaces de Usuario



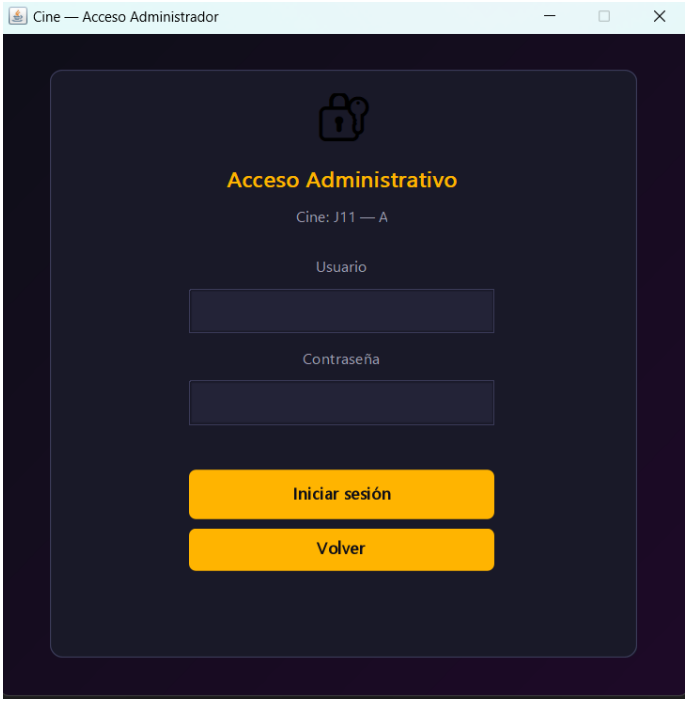
Selección de Sucursal:

En esta pantalla el usuario puede escoger mediante combo box el estado y municipio del cine al cual se quiere acceder, cuando se tiene seleccionado el estado y municipio mencionado, mediante el botón rojo inferior con el texto “Entrar al cine” podrás pasar a lo siguiente.



Tipo de usuario:

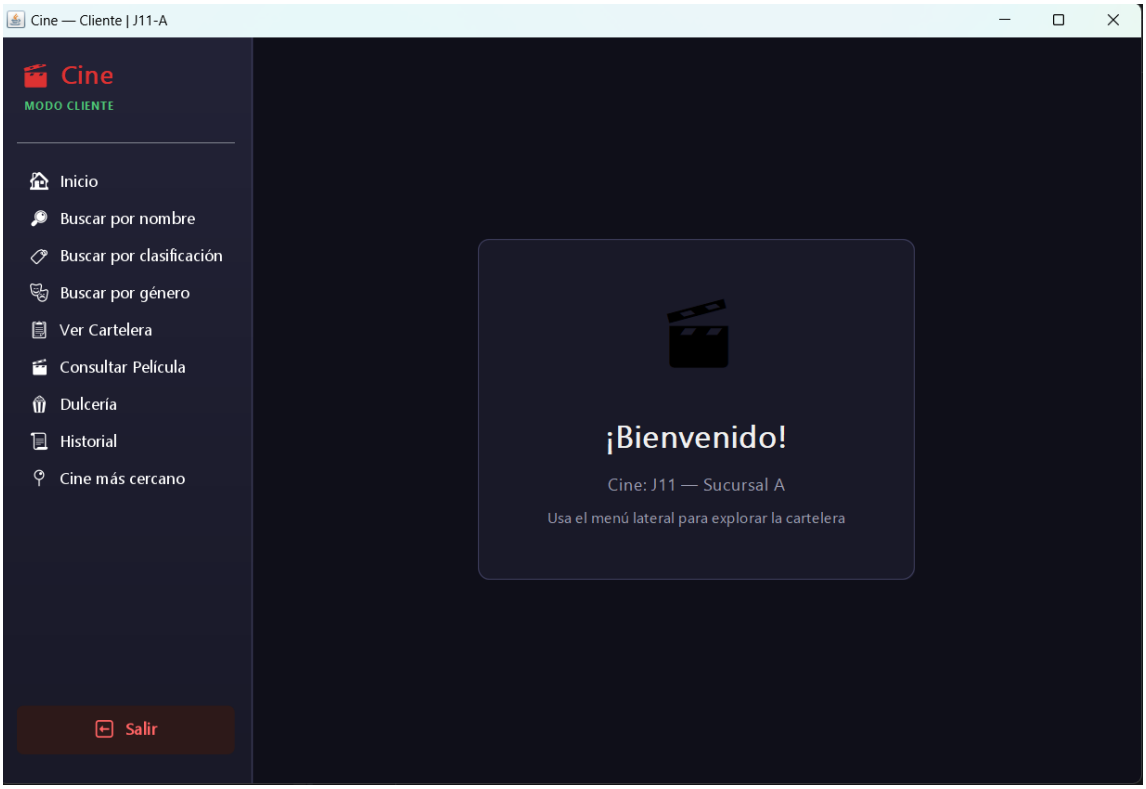
En la pantalla anterior se despliega un JOptionPane con dos opciones en forma de botones, “Administrador” y “Cliente”, dependiendo de cual escojas te lleva a una interfaz u otra.



Acceso administrativo:

En esta pantalla el usuario debe registrarse mediante dos campos, Usuario y Contraseña, este ultimo tiene la particularidad que cuida la visibilidad de la contraseña y cada carácter que el usuario ingresa lo muestra cómo *.

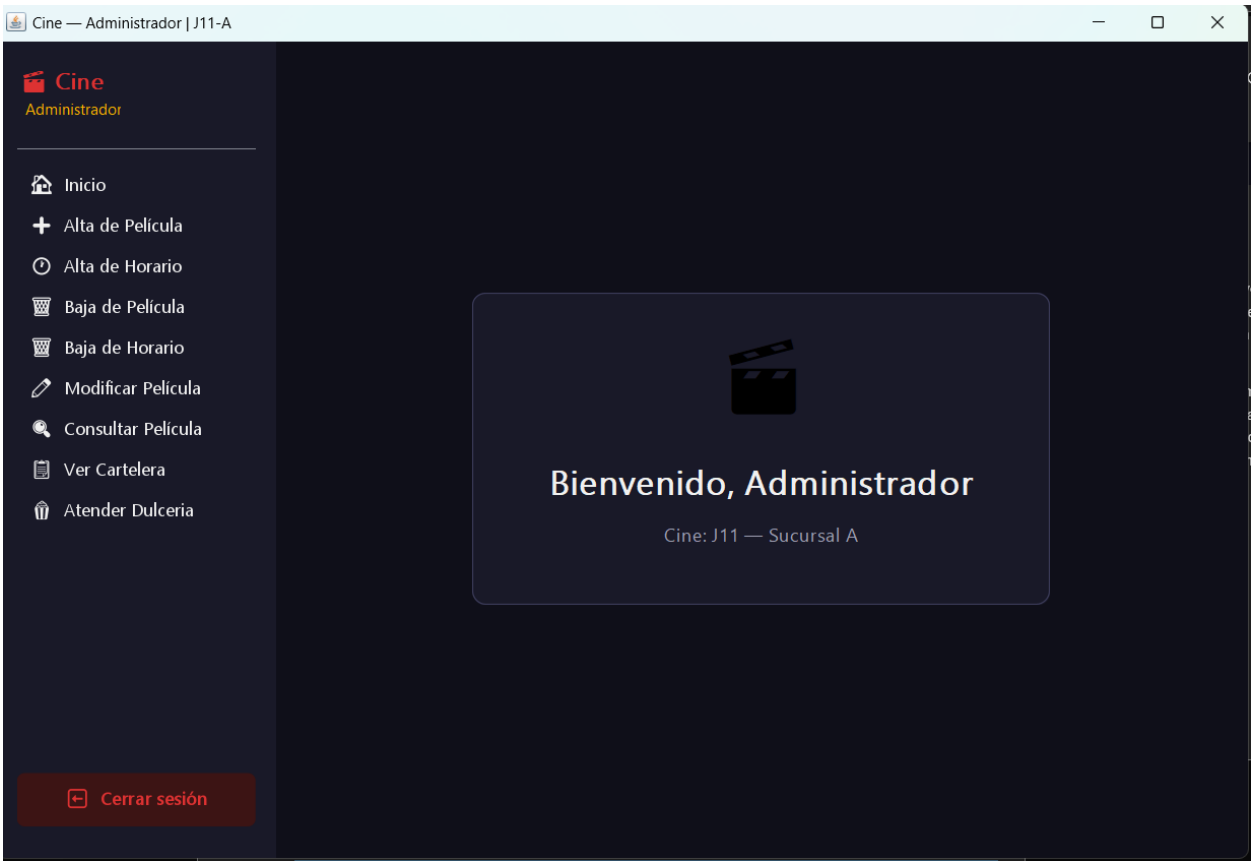
En la parte baja tenemos dos botones, Iniciar sesión que valida los datos que se ponen en los campos de arriba y volver que nos devuelve a la pantalla Selección de sucursal.



Interfaz de Cliente:

En esta pantalla se presenta el panel principal de interacción para el usuario, el cual cuenta con un diseño oscuro y moderno. En la parte izquierda tenemos un menú lateral de navegación que organiza todas las funciones del sistema mediante iconos y etiquetas claras; este incluye opciones para realizar búsquedas personalizadas (por nombre, clasificación o género), visualizar la cartelera completa, acceder a la dulcería y consultar el historial de navegación.

En la zona central se muestra una **ventana de bienvenida** dinámica que confirma la sucursal seleccionada (por ejemplo, Cine: J11 — Sucursal A) e instruye al usuario a utilizar el menú para explorar el contenido. Finalmente, en la esquina inferior izquierda se ubica un botón de Salir, resaltado en color rojo, que permite al usuario finalizar su sesión actual y regresar a la pantalla de selección inicial.



Interfaz de Modo Administrador:

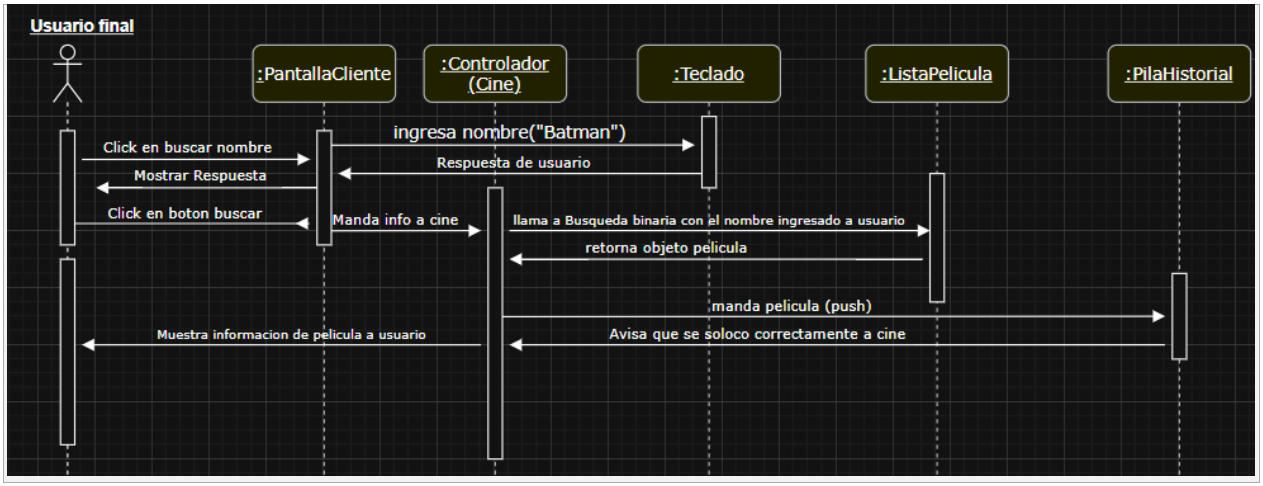
En esta pantalla se visualiza el panel de control exclusivo para el personal autorizado, manteniendo la línea estética de la aplicación en modo oscuro. En la parte izquierda se ubica el **menú de gestión operativa**, el cual despliega una serie de herramientas críticas para el mantenimiento de la sucursal, tales como el alta y baja de películas, la programación de nuevos horarios, la modificación de datos existentes y la opción de "Atender Dulcería" para gestionar la cola de pedidos.

En la zona central se aprecia un **mensaje de bienvenida al administrador**, especificando nuevamente la clave de la sucursal (Cine: J11 — Sucursal A) para evitar errores en la carga de datos. Finalmente, en la parte baja del menú lateral se encuentra el botón **Cerrar sesión**, diseñado en un tono rojizo para facilitar su identificación, el cual permite salir del entorno administrativo y regresar de forma segura al inicio del programa.

8.Diseño de comportamiento

8.1 Flujos de proceso principales

8.1.1 Flujo: Consulta de Cartelera y Gestión de Historia



El diagrama de secuencia representa la interacción entre la capa de presentación y la lógica de negocio. Se usa de un **algoritmo de** búsqueda binaria para la recuperación de datos desde la ListaPelicula. Una vez obtenida la información, el sistema asegura la persistencia temporal realizando un push() en la PilaHistorial, garantizando que el usuario pueda retomar consultas previas.

8.2 Manejo de errores y excepciones

La estrategia general de manejo de errores en el sistema se basa en la prevención de estados nulos y la retroalimentación inmediata al usuario mediante interfaces gráficas.

- **Validación Preventiva:** Antes de realizar cualquier operación sobre estructuras de datos (como pop en la pila o cola), el sistema invoca métodos de comprobación como estaVacia(), evitando excepciones de tipo NullPointerException.
- **Captura de Excepciones (Try-Catch):** Se implementan bloques de captura específicamente en la capa de persistencia (I/O) para gestionar errores de lectura/escritura de archivos .txt, asegurando que, ante la falta de un archivo de sucursal, el sistema no se interrumpa y notifique al administrador.

- **Mensajería al Usuario:** Los errores de lógica de negocio (como "Historial lleno" o "Pedido inexistente") se comunican mediante cuadros de diálogo (JOptionPane) o mensajes en consola, permitiendo que el usuario entienda la restricción sin que la aplicación se cierre inesperadamente.

8.3 Transacciones y concurrencia

Debido a que el sistema está diseñado como una aplicación de escritorio **Standalone** basada en archivos planos, el manejo de transacciones y concurrencia se define bajo los siguientes criterios:

- **Atomicidad en Archivos:** Dado que no existe un motor de base de datos relacional, la "transacción" se asegura mediante el método guardarDatos() de la clase Cine. Este método sobrescribe el archivo .txt de forma completa al final de cada operación crítica, garantizando que el estado en disco coincida con el estado en RAM.
- **Concurrencia Local:** El sistema utiliza el hilo de despacho de eventos de Java (**Event Dispatch Thread**) para gestionar la interfaz gráfica. Esto asegura que todas las actualizaciones de las estructuras de datos (como agregar una película a la lista o un pedido a la cola) se realicen de manera secuencial, evitando condiciones de carrera (*race conditions*) dentro de la misma instancia.
- **Sincronización de Datos:** Al cargar los datos al inicio de la sesión mediante cargarDatos(), el sistema bloquea simbólicamente la información en una instancia única de Cine, asegurando que todas las pantallas (Modo Cliente y Modo Administrador) operen sobre la misma referencia de memoria.

9. Instalación y Configuración

9.1 Requisitos de hardware y software

Componente	Mínimo	Recomendado
CPU	1.6 GHz Dual Core	2.4 GHz Quad Core o superior
RAM	2 GB	4 GB
Almacenamiento	100 MB de espacio libre	500 MB de espacio libre
S.O.	Windows 7 / Linux / macOS	Windows 10/11
Red	No necesaria	No necesaria

9.2 Variables de entorno y configuración

Liste todas las variables de entorno necesarias para el correcto funcionamiento del sistema.

Variable	Valor ejemplo	Descripción
JAVA_HOME	C:\Program Files\Java\jdk-17	Ruta de instalación del JDK/JRE para ejecutar el archivo .jar.
CLASSPATH	.;target/classes	Ruta para que la JVM localice las clases personalizadas del proyecto.
DATA_PATH	./ (Raíz)	Directorio donde el sistema busca los archivos .txt de las sucursales.

9.3 Procedimiento de instalación

1. **Instalación del JRE:** Descargue e instale el Java Runtime Environment (versión 8 o superior) desde el sitio oficial.
2. **Descarga del Paquete:** Copie el archivo ejecutable ProyectoCine.jar en la carpeta de destino deseada.
3. **Preparación de Datos:** Asegúrese de incluir los archivos de texto de las sucursales (ej. J11_A.txt) en la misma carpeta raíz que el archivo .jar.
4. **Ejecución:** Ejecute el sistema con doble clic sobre el archivo o mediante el comando java -jar ProyectoCine.jar en la terminal.

9.4 Procedimiento de actualización

Para actualizar el sistema sin pérdida de información de las sucursales o cartelera:

1. Realice un respaldo de los archivos con extensión **.txt** que contienen la persistencia de datos actual.
2. Reemplace el archivo ProyectoCine.jar antiguo por la nueva versión proporcionada.
3. Al iniciar la nueva versión, el sistema detectará automáticamente los archivos .txt anteriores y cargará la información en las nuevas estructuras de memoria.

9.5 Procedimiento de rollback

En caso de que la nueva versión presente errores de compatibilidad:

1. Elimine el archivo .jar de la versión fallida.
2. Restaure el archivo .jar de la versión anterior desde su copia de seguridad.
3. En caso de corrupción de datos, restaure los archivos **.txt** del respaldo realizado en el paso 9.4 antes de volver a ejecutar la aplicación.

10. Rendimiento y estabilidad

10.1 Métricas de rendimiento

Métrica	Objetivo	Critico	Herramienta de medición
Tiempo de respuesta (p95)	< 500 ms	> 2 s	System.nanoTime() (Java)
Carga de sucursal	< 300 ms	< 99%	[Herramienta]
Heap	< 200 MB	> 512 MB	Task Manager
Complejidad de Ruta (Floyd)	< 50 ms	> 200 ms	Perfilado de métodos (Profiler)

10.2 Estrategia de escalabilidad

Dada la naturaleza del sistema como aplicación de escritorio basada en estructuras de datos dinámicas, la estrategia de escalabilidad se define de la siguiente manera:

- **Escalabilidad Vertical (RAM/CPU):** El sistema está diseñado para escalar verticalmente. Al ser una aplicación que carga toda la cartelera en memoria, un aumento en la memoria RAM permite gestionar un mayor volumen de sucursales y películas simultáneamente sin degradar el tiempo de acceso.
- **Optimización Algorítmica (Cacheo):** Para evitar el recálculo constante de rutas entre cines, el sistema utiliza una estrategia de "cacheo" mediante una matriz de adyacencia estática resultante del algoritmo de Floyd-Warshall. Una vez calculada la matriz al inicio, el acceso a cualquier ruta es de tiempo constante O(1).
- **Escalabilidad de Datos (Fragmentación):** Aunque no existe escalamiento horizontal por red, el sistema escala de forma lógica mediante la fragmentación de archivos. Cada sucursal tiene su propio archivo .txt independiente, lo que evita que el crecimiento de una sucursal afecte el rendimiento de carga de las demás.
- **Eficiencia de Estructuras:** El uso de listas ligadas (NodoPelicula) en lugar de arreglos dinámicos (ArrayList) permite una escalabilidad lineal en la inserción de

datos, eliminando la necesidad de re-dimensionar y copiar arreglos en memoria durante la ejecución.

Nota técnica: En un sistema de escritorio, no aplica el *auto-scaling* o *balanceo de carga* de red, por lo que la justificación se centra en la eficiencia de los algoritmos para manejar más datos con los mismos recursos.

11. Estrategia de Pruebas

11.1 Ambientes de prueba

Dada la arquitectura del sistema, los ambientes se definen por el nivel de aislamiento de los datos de persistencia:

- **Ambiente de Desarrollo (Local):** Entorno donde el desarrollador ejecuta el código directamente desde el IDE (NetBeans/IntelliJ). Se utilizan archivos .txt de prueba con datos ficticios para validar la lógica de las estructuras de datos.
- **Ambiente de Pruebas (QA):** Copia exacta del ejecutable .jar fuera del IDE. En este ambiente se realizan pruebas de "caja negra" para verificar que la interfaz visual (Swing) responda correctamente a las interacciones del usuario y que los diálogos de error aparezcan en los momentos adecuados.
- **Ambiente de Producción:** El entorno final del cliente. En este nivel, el sistema opera con los archivos de texto reales de las sucursales. No se realizan cambios en el código en este ambiente; cualquier error detectado debe ser replicado en el ambiente de desarrollo para su corrección.

12. Mantenimiento y Operacion

12.1 Monitoreo del sistema

Dado que el sistema se ejecuta de forma local (Client-side), el monitoreo se centra en la salud del proceso y la integridad de la memoria:

- **Monitoreo de Recursos:** Se recomienda el uso del Administrador de Tareas del sistema operativo para supervisar el consumo de memoria Heap y asegurar que no existan fugas de memoria tras sesiones prolongadas de uso.
- **Alertas de Integridad:** El sistema incluye validaciones internas que lanzan cuadros de diálogo de alerta (JOptionPane) si se detecta que los archivos .txt de las sucursales están corruptos o no tienen el formato esperado al momento de la carga inicial.

12.2 Gestión de logs

El sistema genera registros de actividad para facilitar la depuración y el seguimiento de operaciones administrativas:

Ubicación: Los registros son redirigidos a un archivo log.txt en la raíz del proyecto.

Niveles de Logging:

INFO: Registro de inicio de sesión, carga exitosa de sucursales y cierres de sesión.

WARNING: Intentos de acceso fallidos al modo administrador o falta de archivos de configuración no críticos.

ERROR: Fallos en la escritura de archivos (IOException) o desbordamiento de estructuras de datos.

12.3 Procedimientos de soporte

Para garantizar la continuidad operativa de las sucursales, se establece el siguiente protocolo:

- **Escalamiento de Incidentes:**
    1. **Nivel 1 (Operador):** Problemas de uso, errores de dedo en formularios o dudas de navegación.

2. **Nivel 2 (Administrador Local):** Problemas con archivos .txt faltantes o permisos de escritura en la carpeta del sistema.
3. **Nivel 3 (Soporte Técnico/Desarrollador):** Errores de lógica en algoritmos (ej. rutas de Floyd incorrectas) o fallos críticos al arrancar el .jar.